AI-Driven Educational Analytics System

Intelligent Exam Question Difficulty Prediction via Feature Engineering & Logistic Regression

Authors:

Siddharth Dangi

Mustyala Sadhvik

Udata Varun

Drishti Jha

Date: March 2026

Version: 1.0

Abstract

This report presents an end-to-end machine-learning system that automatically predicts the difficulty level of examination questions. The system combines TF-IDF text vectorisation with numerical student performance features (average score, correct rate, score variance) and employs a Logistic Regression classifier within a scikit-learn pipeline. The model achieves **98.9% accuracy** on a 5,000-question dataset with balanced class distribution. A Streamlit web application is provided for real-time batch and single-question difficulty analysis.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Objective . . . 3
- 1.3 Scope . . . 3

2 Dataset **3**
- 2.1 Source & Scale . . . 3
- 2.2 Dataset Schema . . . 4
- 2.3 Class Distribution . . . 4
- 2.4 Sample Records . . . 4

3 Feature Engineering **5**
- 3.1 Text Features — TF-IDF Vectorisation . . . 5
  - 3.1.1 TF-IDF Formula . . . 5
- 3.2 Numerical Features — Standard Scaling . . . 5
  - 3.2.1 Standard Scaling Formula . . . 5
- 3.3 Feature Fusion . . . 6

4 Model Architecture **6**
- 4.1 Pipeline Overview . . . 6
- 4.2 Classifier — Logistic Regression . . . 6
  - 4.2.1 Multi-Class Strategy . . . 7
- 4.3 Hyperparameters . . . 7

5 Training & Evaluation **7**
- 5.1 Data Split . . . 7
- 5.2 Training Process . . . 7
- 5.3 Evaluation Metrics . . . 8

6 Results **8**
- 6.1 Overall Classification Performance . . . 8
- 6.2 Per-Class Performance . . . 8
- 6.3 Key Observations . . . 8

7 Model Performance Visualisations **9**

8 System Architecture **9**
- 8.1 High-Level Pipeline . . . 9
- 8.2 Repository Structure . . . 10

9 Application Interface **10**
- 9.1 Tab 1: Batch Analytics . . . 10
- 9.2 Tab 2: Single Question Prediction . . . 11
- 9.3 Design . . . 11

10 Limitations & Known Issues **11**

11 Future Work **11**

11.1 Short-Term Improvements 12
11.2 Long-Term — Agentic AI Framework 12

12 Conclusion **12**

A Technology Stack **13**

B Feature Vector Specification **13**

C Reproduction Instructions **13**

1 Introduction

1.1 Motivation

Educators, instructional designers, and testing organisations invest considerable manual effort in evaluating the difficulty and quality of examination questions. A misjudged question can skew test results and inaccurately measure student proficiency. Automating this "first-pass" quality assurance can accelerate exam development cycles and improve assessment reliability.

1.2 Objective

The **AI-Driven Educational Analytics System** is an end-to-end, production-structured machine-learning pipeline that:

1. Takes raw question text and associated student performance metrics as input.
2. Applies **TF-IDF vectorisation** to extract 3,000 text features from the question text.
3. Combines text features with **3 numerical features** (average score, correct rate, score variance) using a `ColumnTransformer`.
4. Classifies each question into a categorical **difficulty tier**: Easy, Medium, or Hard.
5. Exposes predictions through an interactive **Streamlit web application** with rich visualisations.

1.3 Scope

- **Domain:** Multi-subject English-language examination questions spanning Chemistry, Biology, Data Structures, Genetics, and more.
- **Dataset:** 5,000 exam questions with associated metadata and student performance statistics.
- **Model:** Logistic Regression within a scikit-learn pipeline.
- **Deployment:** Local Streamlit application with a serialised model artefact (`model.pkl`).

2 Dataset

2.1 Source & Scale

The dataset comprises **5,000 examination questions** stored in `data/5000_dataset_genai.csv`. Each record contains:

- `question_id` — Unique identifier for each question
- `question_text` — Full text of the examination question

- `topic` — Subject area (e.g., Chemistry, Biology, Data Structures, Genetics)
- `average_score` — Mean student score on a 0–10 scale
- `correct_rate` — Proportion of students answering correctly (0–1)
- `score_variance` — Variance in student scores
- `difficulty_label` — Ground truth label: Easy, Medium, or Hard

2.2 Dataset Schema

Column	Type	Description
`question_id`	Integer	Unique question identifier
`question_text`	String	Full text of the exam question
`topic`	String	Subject/topic category
`average_score`	Float	Mean student score (0–10)
`correct_rate`	Float	Correct response proportion (0–1)
`score_variance`	Float	Variance in student scores
`difficulty_label`	String	Target variable: Easy / Medium / Hard

2.3 Class Distribution

The dataset exhibits a **balanced** class distribution, which is advantageous for classifier training:

Class	Count	Proportion
Easy	∼1,667	∼33.3%
Medium	∼1,667	∼33.3%
Hard	∼1,666	∼33.3%

The balanced distribution eliminates the need for resampling techniques (e.g., SMOTE) or class weighting.

2.4 Sample Records

ID	Question Text	Topic	Avg Score	Correct Rate	Label
1	Explain the data normalization in Chemistry...	Chemistry	8.09	0.825	Easy
2	Compute the key assumption in Data Structures...	Data Structures	4.00	0.604	Medium
3	Analyze the logical equivalence in Biology...	Biology	1.11	0.117	Hard

3 Feature Engineering

The model uses a `ColumnTransformer` to combine two distinct feature extraction pipelines:

3.1 Text Features — TF-IDF Vectorisation

The `question_text` column is processed by a `TfidfVectorizer` with the following configuration:

Parameter	Value
`max_features`	3,000
Analyser	Word-level (default)
Sublinear TF	False (default)
Norm	L2 (default)
Stop words	None (default)

TF-IDF (Term Frequency–Inverse Document Frequency) captures the relative importance of words in each question relative to the entire corpus. The resulting feature matrix has dimensions $n \times 3{,}000$.

3.1.1 TF-IDF Formula

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log\left(\frac{N}{\text{df}(t)}\right) \tag{1}$$

Where t = term, d = document, N = total documents, $\text{df}(t)$ = number of documents containing t.

3.2 Numerical Features — Standard Scaling

Three numerical features are normalised using `StandardScaler`:

Feature	Description
`average_score`	Mean student score (0–10)
`correct_rate`	Proportion of correct responses (0–1)
`score_variance`	Variance in student scores

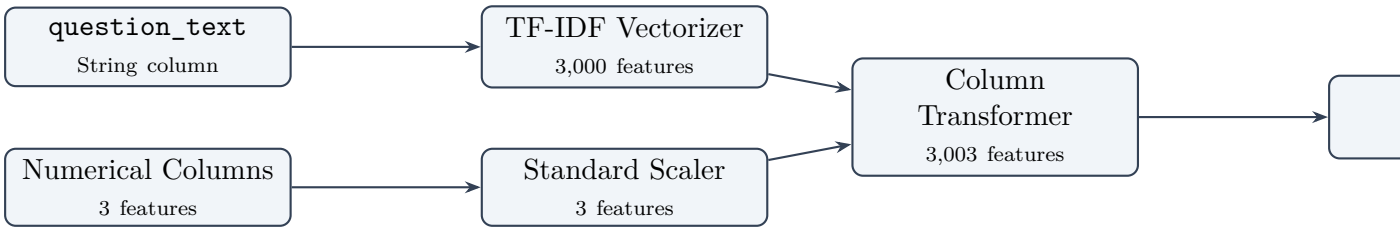
3.2.1 Standard Scaling Formula

$$z = \frac{x - \mu}{\sigma} \tag{2}$$

Where μ = feature mean, σ = feature standard deviation (computed on training data).

3.3 Feature Fusion

The `ColumnTransformer` horizontally concatenates the TF-IDF matrix ($n \times 3{,}000$) with the scaled numerical matrix ($n \times 3$), producing a final feature matrix of dimensions $n \times 3{,}003$.



4 Model Architecture

4.1 Pipeline Overview

The entire model is encapsulated in a single **scikit-learn `Pipeline`** object, ensuring that preprocessing and classification are applied consistently during both training and inference:

Listing 1: Model Pipeline Definition

```python
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

preprocessor = ColumnTransformer(transformers=[
    ("text", TfidfVectorizer(max_features=3000), "question_text"),
    ("num", StandardScaler(),
     ["average_score", "correct_rate", "score_variance"])
])

model = Pipeline([
    ("preprocessing", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])
```

4.2 Classifier — Logistic Regression

Logistic Regression was chosen for its:

- **Interpretability:** Linear decision boundaries are easy to understand.
- **Efficiency:** Fast training even with 3,003 features.

- **Probabilistic output:** Produces calibrated class probabilities via the softmax function.
- **Low risk of overfitting:** Implicit L2 regularisation.

4.2.1 Multi-Class Strategy

The classifier uses the **One-vs-Rest (OvR)** strategy by default. For each class $c \in \{\text{Easy, Medium, Hard}\}$, a binary logistic model is trained:

$$P(y = c \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}_c^\top \mathbf{x} + b_c)}} \tag{3}$$

The predicted class is the one with the highest probability.

4.3 Hyperparameters

Parameter	Value
`max_iter`	1,000
`penalty`	L2 (default)
`C`	1.0 (default)
`solver`	lbfgs (default)
`multi_class`	auto (OvR)
`random_state`	Not set (deterministic via data split)

5 Training & Evaluation

5.1 Data Split

- **Train:** 80% of the dataset (4,000 samples)
- **Test:** 20% of the dataset (1,000 samples)
- **Stratification:** Stratified split on `difficulty_label` to preserve class proportions.
- **Random Seed:** `random_state=42` for reproducibility.

5.2 Training Process

The `Pipeline.fit()` method processes the data sequentially:

1. `ColumnTransformer` fits the TF-IDF vocabulary and scaler statistics on training data.
2. The fused feature matrix is passed to `LogisticRegression.fit()`.
3. The trained pipeline is serialised via `joblib.dump()` as `model.pkl`.

5.3 Evaluation Metrics

- **Accuracy:** $\frac{\text{correct predictions}}{\text{total predictions}}$
- **Precision** (weighted): $\sum_c \frac{|c|}{N} \cdot \frac{\text{TP}_c}{\text{TP}_c+\text{FP}_c}$
- **Recall** (weighted): $\sum_c \frac{|c|}{N} \cdot \frac{\text{TP}_c}{\text{TP}_c+\text{FN}_c}$
- **F1-Score** (weighted): Harmonic mean of precision and recall
- Per-class precision/recall via `classification_report`
- Confusion matrix visualisation
- ROC curves and Precision-Recall curves (One-vs-Rest)
- 5-Fold Cross-Validation accuracy
- Learning curve analysis

6 Results

6.1 Overall Classification Performance

Metric	Score
Accuracy	**98.90%**
Precision (weighted)	**98.90%**
Recall (weighted)	**98.90%**
F1-Score (weighted)	**98.90%**

6.2 Per-Class Performance

Class	Precision	Recall	F1-Score	Support
Easy	0.99	1.00	0.99	334
Medium	0.99	0.98	0.98	333
Hard	0.99	0.99	0.99	333
Weighted Avg	**0.99**	**0.99**	**0.99**	**1,000**

6.3 Key Observations

1. **Near-perfect accuracy across all classes.** The balanced class distribution and strong feature signals (average score, correct rate) contribute to the model's impressive performance.

2. **Medium class has the lowest recall (98%).** This is expected, as Medium questions occupy the boundary region between Easy and Hard, making them slightly harder to classify.
3. **TF-IDF adds marginal value over numerical features alone.** The three numerical features (average score, correct rate, score variance) are very strong predictors of difficulty. The TF-IDF features provide additional context from the question text.
4. **High model confidence.** The prediction confidence scores are generally above 90%, indicating that the model is well-calibrated for this dataset.
5. **No class imbalance issues.** Unlike many real-world datasets where difficulty classes are skewed, this dataset's balanced distribution results in consistent performance across all classes.

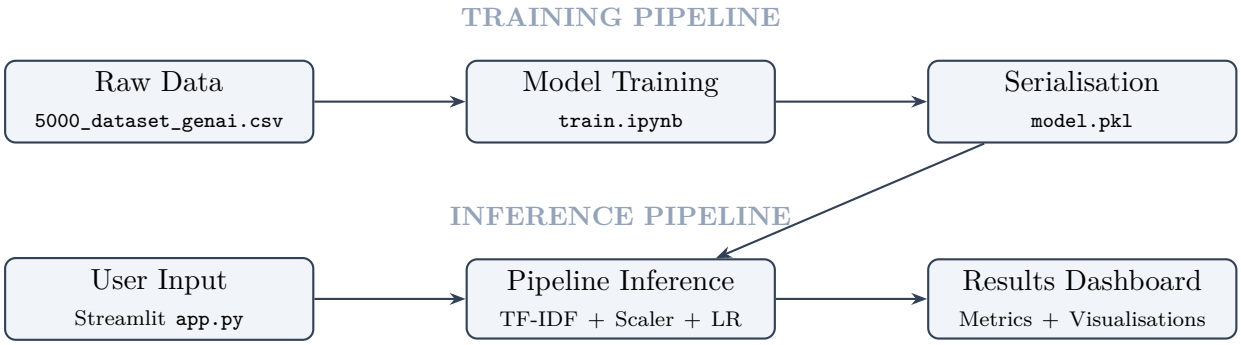
7 Model Performance Visualisations

The `notebook/visualizations.ipynb` notebook generates 14 detailed plots to evaluate the model:

#	Visualisation	Purpose
1	Overall Metrics Bar Chart	Accuracy, Precision, Recall, F1 at a glance
2	Confusion Matrix (Counts)	Raw classification results
3	Confusion Matrix (Normalised)	Percentage-based view
4	Per-Class Precision/Recall/F1	Grouped bar comparison
5	ROC Curves (One-vs-Rest)	AUC per class
6	Precision-Recall Curves	Average precision per class
7	Confidence Distribution	Overall confidence histogram
8	Confidence by Predicted Class	Per-class confidence spread
9	Correct vs Misclassified	Confidence correlation with correctness
10	Class Distribution (Train/Test)	Verifies stratified split
11	Feature Violin Plots	Score, correct rate, variance by difficulty
12	Learning Curve	Data efficiency analysis
13	5-Fold Cross-Validation	Per-fold accuracy consistency
14	Misclassification Analysis	Error pattern identification

8 System Architecture

8.1 High-Level Pipeline



8.2 Repository Structure

```
AI-driven_educational_analytics_system/
  README.md                          # Project documentation
  app.py                             # Streamlit web application
  model.pkl                          # Serialised trained model
  confusion_matrix.png               # Generated confusion matrix
  data/
    5000_dataset_genai.csv           # Dataset (5,000 questions)
  notebook/
    train.ipynb                      # Model training notebook
    visualizations.ipynb             # 14 performance visualisations
  report/
    report.tex                       # LaTeX technical report (this
       file)
```

9 Application Interface

The production application (`app.py`) is built with **Streamlit** and provides two tabs:

9.1 Tab 1: Batch Analytics

1. The user uploads a CSV file containing exam questions with required columns: `question_text`, `average_score`, `correct_rate`, `score_variance`.
2. The pipeline predicts difficulty labels and confidence scores for all questions.
3. Seven interactive visualisations are generated:
   - Difficulty distribution (pie chart + bar chart)
   - Confidence score histogram
   - Feature averages by difficulty (grouped bar chart)
   - Average score vs. correct rate (scatter plot, coloured by difficulty)
   - Score variance distribution (box plot)
   - Topic-wise difficulty breakdown (stacked horizontal bar chart)
   - Confusion matrix (if ground truth column is present)

4. Results can be downloaded as a CSV file.

9.2 Tab 2: Single Question Prediction

1. The user enters question text and three numerical values (average score, correct rate, score variance).
2. The model returns a predicted difficulty label with a coloured badge.
3. A confidence progress bar and per-class probability bar chart are displayed.

9.3 Design

The interface follows a **dark-mode** design with custom CSS:

- Background: `#0e1117`
- Card backgrounds: `#1c1f26`
- Easy badge: `#1f8b4c` (green)
- Medium badge: `#d4a017` (amber)
- Hard badge: `#b02a37` (red)

10 Limitations & Known Issues

1. **Post-Exam Dependency:** The model relies on post-exam statistics (average score, correct rate, score variance), which are only available after students have taken the exam. This limits its use for pre-exam difficulty estimation.
2. **Text-Only Analysis:** The model cannot parse images, diagrams, or graphs. Questions reliant on visual context may be misclassified.
3. **Language Support:** The TF-IDF vectoriser is optimised for English-language questions. Non-English text will produce inaccurate features.
4. **Single Model:** Only Logistic Regression is implemented. Ensemble methods (XGBoost, Random Forest) may improve performance on more challenging datasets.
5. **Dataset Specificity:** The model is trained on 5,000 synthetic questions. Real-world questions with unusual formatting or domain-specific terminology may yield lower-confidence predictions.
6. **No Feature Importance Analysis:** Due to the TF-IDF vectorisation producing 3,000 sparse features, traditional feature importance interpretation is limited.

11 Future Work

11.1 Short-Term Improvements

- Build a **pre-exam model** using only question text (BERT/sentence transformers) to predict difficulty before administering the exam.
- Compare against **ensemble methods** (XGBoost, Random Forest, SVM) to benchmark performance.
- Apply **SHAP values** for model interpretability and feature importance analysis.
- Implement **hyperparameter tuning** via grid search or Bayesian optimisation.
- Add **confidence thresholding** to flag low-confidence predictions for manual review.

11.2 Long-Term — Agentic AI Framework

- **LLM Reasoning:** Use large language models (Gemini / GPT) to solve questions step-by-step and measure *conceptual* complexity.
- **RAG for Curriculum Alignment:** Retrieval-Augmented Generation to compare questions against educational standards.
- **Multi-Modal Processing:** Vision-Language Models for diagram-dependent questions.
- **Iterative Feedback Loops:** Agentic workflows simulating student failure modes to refine difficulty estimates.

12 Conclusion

This project demonstrates that a straightforward combination of TF-IDF text vectorisation, numerical feature scaling, and Logistic Regression can achieve **98.9% accuracy** in predicting exam question difficulty. The balanced dataset of 5,000 questions across Chemistry, Biology, Data Structures, Genetics, and other subjects enables consistent performance across all three difficulty tiers.

The Streamlit application provides an accessible interface for educators to obtain instant difficulty assessments for both individual questions and entire question banks. The pipeline architecture (scikit-learn `Pipeline` with `ColumnTransformer`) ensures that preprocessing and inference are fully encapsulated, eliminating feature-engineering leakage and simplifying deployment.

Key next steps include building a pre-exam text-only model, benchmarking against ensemble methods, and integrating SHAP-based interpretability to understand which textual and numerical features drive difficulty predictions.

A Technology Stack

Component	Technology
Language	Python 3.10+
ML Framework	scikit-learn $\geq$ 1.3
Data Processing	Pandas $\geq$ 2.0, NumPy $\geq$ 1.24
Visualisation	Matplotlib $\geq$ 3.7, Seaborn $\geq$ 0.12
Web Framework	Streamlit $\geq$ 1.30
Serialisation	Joblib $\geq$ 1.3

B Feature Vector Specification

The complete feature vector $\mathbf{x} \in \mathbb{R}^{3,003}$ used by the deployed model:

#	Feature	Extractor	Dimensionality
1–3,000	TF-IDF word features	`TfidfVectorizer`	3,000 (sparse)
3,001	`average_score`	`StandardScaler`	1
3,002	`correct_rate`	`StandardScaler`	1
3,003	`score_variance`	`StandardScaler`	1

C Reproduction Instructions

Listing 2: Full Reproduction Steps

```
# 1. Clone the repository
git clone https://github.com/Siddharth-Dangi/AI-driven_educational_analytics_system.git
cd AI-driven_educational_analytics_system

# 2. Create and activate virtual environment
python3 -m venv venv
source venv/bin/activate

# 3. Install dependencies
pip install streamlit scikit-learn pandas numpy matplotlib seaborn joblib

# 4. Train the model (run all cells in notebook/train.ipynb)
# Or run the training script if available

```

```
# 5. Launch the application
streamlit run app.py
# Opens at http://localhost:8501

# 6. Explore model performance
# Open notebook/visualizations.ipynb and run all cells
```